

Java 11 — Standard HTTP Client

Java Version: 11

Year: 2018

Feature: Standard HTTP Client

UniqueClassName: HttpClient

Definition

The **Standard HTTP Client** was standardized in **Java 11** through the `java.net.http` package.

It provides a modern API for sending HTTP requests and receiving HTTP responses without requiring third-party libraries such as Apache HttpClient.

It supports:

- HTTP/1.1
- HTTP/2
- Synchronous requests
- Asynchronous requests using `CompletableFuture`
- GET, POST, PUT, DELETE, etc.
- Request and response bodies
- Configurable timeouts
- HTTP headers

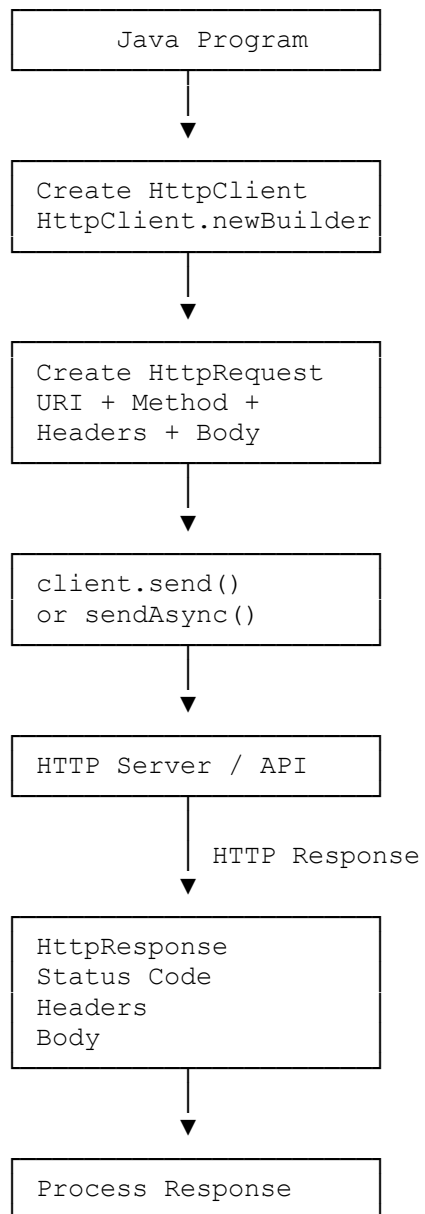
Main Classes

Class	Purpose
HttpClient	Creates and configures an HTTP client
HttpRequest	Represents an HTTP request
HttpResponse	Represents the server response
HttpResponse.BodyHandlers	Specifies how the response body is processed

The package is:

```
import java.net.http.*;
```

Flowchart



Java Code Example — GET Request

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class HttpClient {

    public static void main(String[] args) throws Exception {
```

```

// 1. Create HTTP Client
java.net.http.HttpClient client =
    java.net.http.HttpClient.newHttpClient();

// 2. Create HTTP Request
HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://httpbin.org/get"))
    .GET()
    .build();

// 3. Send Request
HttpResponse<String> response =
    client.send(
        request,
        HttpResponse.BodyHandlers.ofString()
    );

// 4. Process Response
System.out.println("Status Code: "
    + response.statusCode());

System.out.println("Response Body:");
System.out.println(response.body());
}
}

```

Expected Output

Status Code: 200

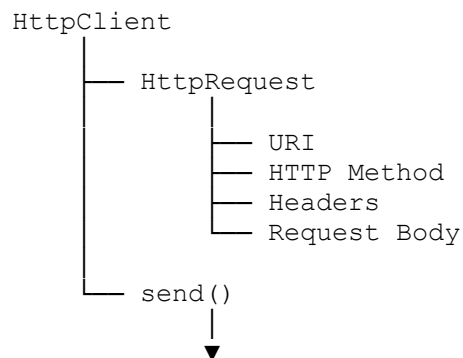
Response Body:

```

{
  "args": {},
  "headers": {
    ...
  },
  "origin": "...",
  "url": "https://httpbin.org/get"
}

```

Important API Flow



```
HttpResponse
├── statusCode()
├── headers()
└── body()
```

Asynchronous Example

Java 11 also allows non-blocking HTTP requests with `sendAsync()`:

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class HttpClientAsyncExample {

    public static void main(String[] args) {

        HttpClient client = HttpClient.newHttpClient();

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("https://httpbin.org/get"))
            .GET()
            .build();

        client.sendAsync(
            request,
            HttpResponse.BodyHandlers.ofString()
        )
        .thenApply(HttpResponse::body)
        .thenAccept(System.out::println)
        .join();
    }
}
```

Here, `sendAsync()` returns a `CompletableFuture`, allowing the program to continue without waiting synchronously for the HTTP response.

In short

Java 11 Standard HTTP Client = `HttpClient` + `HttpRequest` + `HttpResponse`

It replaced many common use cases for older HTTP APIs with a modern, built-in HTTP client supporting **HTTP/1.1**, **HTTP/2**, **synchronous and asynchronous communication**.

Java 11 — String New Methods

Java Version: 11

Year: 2018

Feature: String New Methods

UniqueClassName: StringMethods

Definition

Java 11 introduced several useful methods to the `java.lang.String` class to make common string-processing operations simpler and more readable.

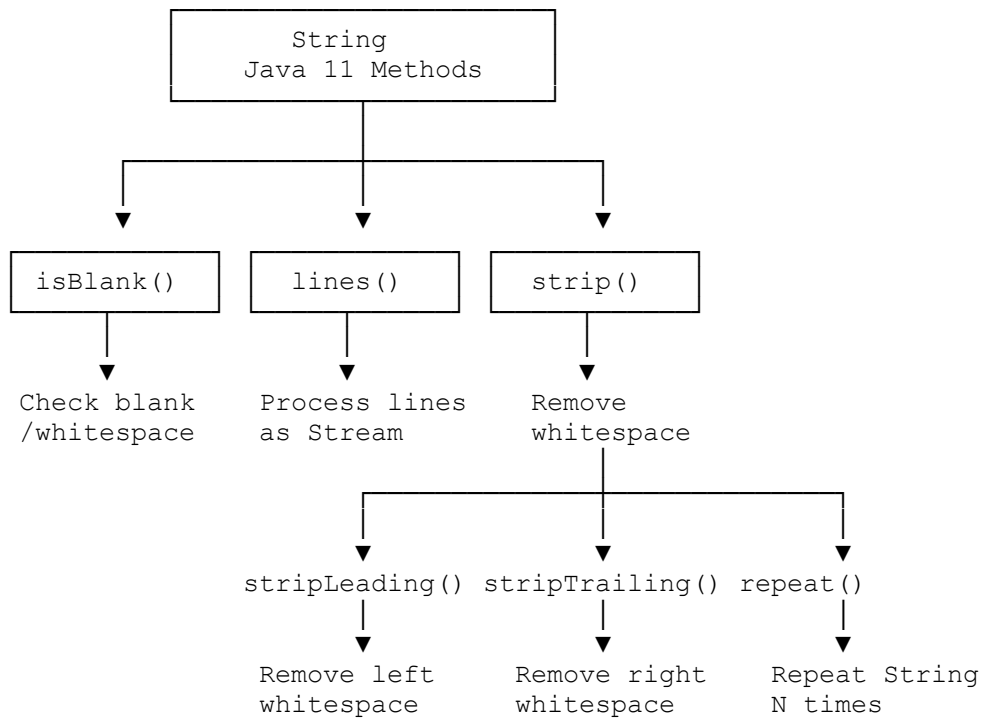
The most notable new methods are:

Method	Purpose	Example
isBlank()	Checks whether a string is empty or contains only whitespace	<code>" ".isBlank() → true</code>
lines()	Splits a string into a stream of lines	<code>"A\nB".lines()</code>
strip()	Removes leading and trailing Unicode whitespace	<code>" Hello ".strip()</code>
stripLeading()	Removes leading Unicode whitespace	<code>" Hello".stripLeading()</code>
stripTrailing()	Removes trailing Unicode whitespace	<code>"Hello ".stripTrailing()</code>
repeat(int)	Repeats a string a specified number of times	<code>"Java".repeat(3) → "JavaJavaJava"</code>

Why were these methods introduced?

Before Java 11, developers commonly used methods such as `trim()`, `split()`, or manual loops for these operations. Java 11 provides more expressive APIs, particularly with better **Unicode whitespace handling** through `strip()`.

Flowchart



Java Code Example

```
import java.util.stream.Stream;

public class StringMethods {

    public static void main(String[] args) {

        // 1. isBlank()
        String blankText = "   ";

        System.out.println("isBlank(): "
            + blankText.isBlank());

        // 2. lines()
        String text = "Java\nPython\nSpring";

        System.out.println("\nlines():");

        text.lines()
            .forEach(System.out::println);

        // 3. strip()
        String message = "   Hello Java   ";

        System.out.println("\nstrip():");
```

```

        System.out.println("'" + message.strip() + "'");

        // 4. stripLeading()
        System.out.println("\nstripLeading():");
        System.out.println("'" + message.stripLeading() + "'");

        // 5. stripTrailing()
        System.out.println("\nstripTrailing():");
        System.out.println("'" + message.stripTrailing() + "'");

        // 6. repeat()
        String java = "Java ";

        System.out.println("\nrepeat():");
        System.out.println(java.repeat(3));
    }
}

```

Expected Output

```
isBlank(): true
```

```
lines():
```

```
Java
Python
Spring
```

```
strip():
'Hello Java'
```

```
stripLeading():
'Hello Java '
```

```
stripTrailing():
' Hello Java'
```

```
repeat():
Java Java Java
```

Method-by-Method Explanation

1. isBlank()

Checks whether the string is empty or contains only whitespace.

```

System.out.println("").isBlank());    // true
System.out.println(" ".isBlank());    // true
System.out.println("Java".isBlank());  // false

```

Unlike simply checking `isEmpty()`, `isBlank()` also recognizes whitespace-only strings.

2. `lines()`

Returns a `Stream<String>` containing the lines of the string.

```
String data = "Java\nSpring\nDocker";
```

```
data.lines()  
    .forEach(System.out::println);
```

Output:

```
Java  
Spring  
Docker
```

This is especially useful when processing multiline text.

3. `strip()`

Removes leading and trailing whitespace.

```
String value = " Java ";
```

```
System.out.println(value.strip());
```

Output:

```
Java
```

`strip()` is Unicode-aware, whereas the older `trim()` uses a more limited definition of whitespace.

4. `stripLeading()`

Removes whitespace only from the beginning.

```
String value = " Java ";
```

```
System.out.println("'" + value.stripLeading() + "'");
```

Output:

'Java '

5. stripTrailing()

Removes whitespace only from the end.

```
String value = " Java ";
```

```
System.out.println("'" + value.stripTrailing() + "'");
```

Output:

' Java'

6. repeat()

Repeats a string a specified number of times.

```
String separator = "-";
```

```
System.out.println(separator.repeat(10));
```

Output:

This is useful for generating separators, formatting output, and creating repeated text.

Key Point

Java 11 String Enhancements

—	isBlank()	→ Check blank/whitespace
—	lines()	→ Process multiline strings
—	strip()	→ Remove both-side whitespace
—	stripLeading()	→ Remove beginning whitespace
—	stripTrailing()	→ Remove ending whitespace
—	repeat()	→ Repeat string N times

These methods made everyday **string validation, whitespace handling, multiline processing, and text repetition** considerably cleaner in Java 11.

Java 11 — Files.readString() / Files.writeString()

Java Version: 11

Year: 2018

Feature: Files.readString() / Files.writeString()

UniqueClassName: FilesStringIO

Definition

Java 11 enhanced the java.nio.file.Files class with two convenient methods for reading and writing text files:

- **Files.readString(Path)** — reads the complete contents of a text file into a String.
- **Files.writeString(Path, CharSequence)** — writes a string to a file.

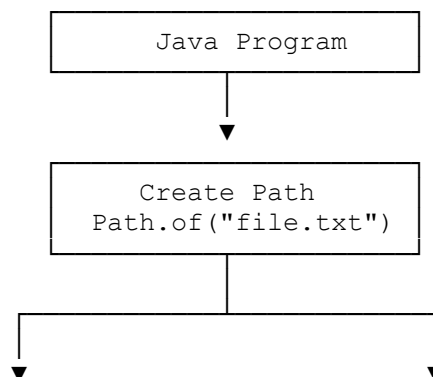
These methods simplify common text-file operations that previously required Files.readAllBytes() with character-set conversion or a BufferedReader/BufferedWriter.

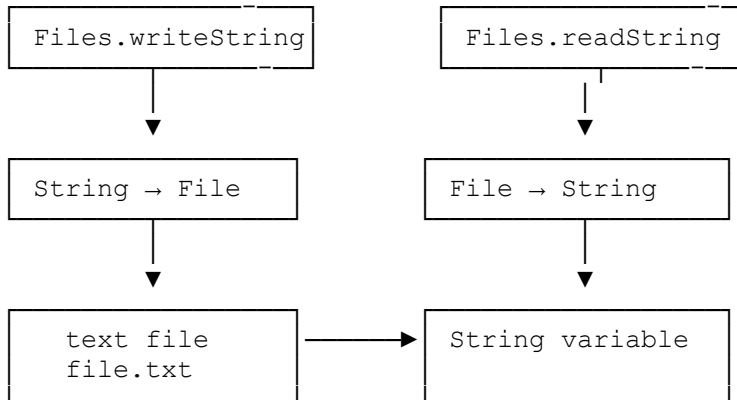
Main Methods

Method	Purpose
Files.readString(path)	Reads an entire text file as a String
Files.readString(path, charset)	Reads using a specified character set
Files.writeString(path, text)	Writes text to a file
Files.writeString(path, text, charset)	Writes using a specified character set
Files.writeString(path, text, options...)	Writes with options such as APPEND

The default charset for these methods is **UTF-8**.

Flowchart





Java Code Example

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;

public class FilesStringIO {

    public static void main(String[] args) {

        Path path = Path.of("java11.txt");

        try {

            // 1. Write String to file
            String content = ""
                Java 11
                Files.readString()
                Files.writeString()
                "";

            Files.writeString(path, content);

            System.out.println("File written successfully.");

            // 2. Read complete file into String
            String result = Files.readString(path);

            System.out.println("\nFile Content:");
            System.out.println(result);

        } catch (IOException e) {
            e.printStackTrace();
        }

    }

}
```

Expected Output

File written successfully.

File Content:

Java 11

Files.readString()

Files.writeString()

The file java11.txt will be created in the program's current working directory.

Append Text Example

`writeString()` can also append content by using `StandardOpenOption.APPEND`.

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;

public class FilesStringIO {

    public static void main(String[] args) {

        Path path = Path.of("data.txt");

        try {

            Files.writeString(
                path,
                "First line\n"
            );

            Files.writeString(
                path,
                "Second line\n",
                StandardOpenOption.APPEND
            );

            String content = Files.readString(path);

            System.out.println(content);

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Output:

First line

Second line

Using a Specific Charset

You can explicitly specify a character set:

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;

public class FilesStringIO {

    public static void main(String[] args) throws IOException {

        Path path = Path.of("unicode.txt");

        String text = "Hello Java 11 — नमस्ते";

        Files.writeString(
            path,
            text,
            StandardCharsets.UTF_8
        );

        String result = Files.readString(
            path,
            StandardCharsets.UTF_8
        );

        System.out.println(result);
    }
}
```

Output:

Hello Java 11 — नमस्ते

Before Java 11

A common approach was:

```
byte[] bytes = Files.readAllBytes(path);
String content = new String(bytes, StandardCharsets.UTF_8);
```

Java 11 simplifies this to:

```
String content = Files.readString(path);
```

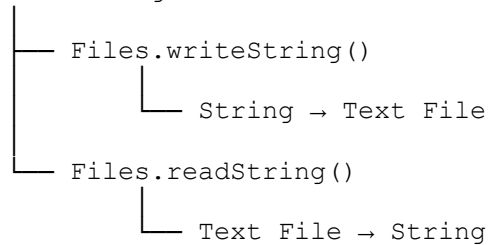
Similarly, writing becomes:

```
Files.writeString(path, "Hello Java 11");
```

instead of manually converting the string to bytes.

Key Point

Java 11 Files String I/O



In short: `Files.readString()` and `Files.writeString()` provide a concise way to perform **whole-file text reading and writing** in Java 11, with UTF-8 as the default charset.

Java 11 — Flight Recorder (JFR)

Java Version: 11

Year: 2018

Feature: Flight Recorder

UniqueClassName: FlightRecorder

Definition

Java Flight Recorder (JFR) is a low-overhead **JVM event-recording and diagnostics framework** used to collect detailed information about a running Java application.

It can record events related to:

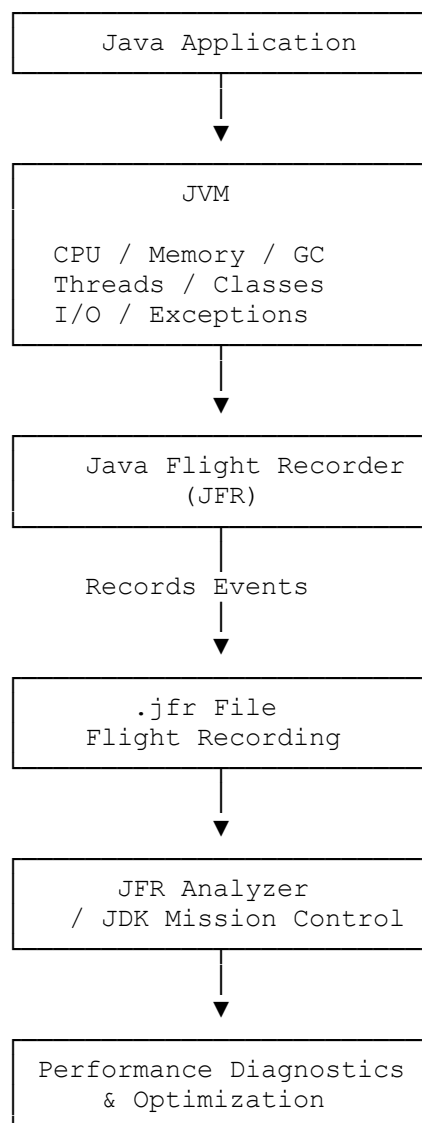
- CPU and thread activity
- Garbage collection
- Memory usage
- Class loading
- Exceptions
- I/O operations

- Locks and synchronization
- JVM behavior
- Application-defined events

The recorded information can later be analyzed to identify **performance bottlenecks, excessive CPU usage, garbage-collection problems, thread contention, and other runtime issues.**

Java 11 made Flight Recorder available as an **OpenJDK feature**, whereas it had previously been associated with Oracle JDK commercial features.

Flight Recorder Architecture



Java Code Example

Java 11 provides the `jdk.jfr` API for programmatic recording.

```
import jdk.jfr.Recording;

import java.nio.file.Path;

public class FlightRecorder {

    public static void main(String[] args) throws Exception {

        // Create a JFR recording
        try (Recording recording = new Recording()) {

            // Start recording
            recording.start();

            System.out.println("Flight Recorder started.");

            // Simulate application activity
            performWork();

            // Stop recording
            recording.stop();

            // Save recording to a JFR file
            Path file = Path.of("application-recording.jfr");

            recording.dump(file);

            System.out.println(
                "Recording saved to: " + file.toAbsolutePath()
            );
        }
    }

    private static void performWork() throws InterruptedException {

        for (int i = 0; i < 5; i++) {

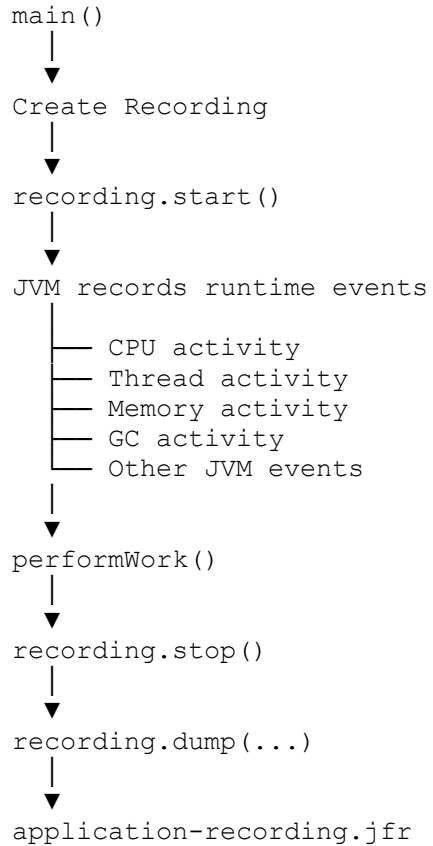
            // Simulate CPU/application work
            long result = 0;

            for (long j = 0; j < 10_000_000; j++) {
                result += j;
            }

            System.out.println("Work completed: " + result);

            Thread.sleep(500);
        }
    }
}
```

Execution Flow



Starting JFR from the Command Line

JFR can also be controlled using JVM options.

For example:

```
java -XX:StartFlightRecording=filename=recording.jfr,duration=60s MyApplication
```

This tells the JVM to:

1. Start Flight Recorder.
2. Record events while MyApplication runs.
3. Record for approximately 60 seconds.
4. Save the recording as recording.jfr.

A recording can then be examined with **JDK Mission Control (JMC)** or other JFR-compatible analysis tools.

Custom Application Events

JFR also allows applications to define their own events.

```
import jdk.jfr.Event;
import jdk.jfr.Label;

public class FlightRecorder {

    @Label("Customer Processing")
    static class CustomerEvent extends Event {

        @Label("Customer ID")
        String customerId;
    }

    public static void main(String[] args) {

        CustomerEvent event = new CustomerEvent();

        event.customerId = "C101";

        event.begin();

        processCustomer();

        event.end();

        if (event.shouldCommit()) {
            event.commit();
        }

        System.out.println("Custom JFR event recorded.");
    }

    private static void processCustomer() {

        for (int i = 0; i < 1_000_000; i++) {
            Math.sqrt(i);
        }
    }
}
```

This allows developers to add **business/application-specific events** to JFR recordings.

Important JFR Classes

Class	Purpose
Recording	Starts, stops, configures, and saves recordings
Event	Base class for custom application events

FlightRecorder	Provides access to the Flight Recorder system
RecordingState	Represents the state of a recording

Package:

```
import jdk.jfr.*;
```

Key Advantages

1. Low Overhead

JFR is designed to collect extensive runtime information while keeping performance overhead relatively low.

2. JVM-Level Visibility

It provides information that ordinary application logging may not reveal, such as GC pauses, thread contention, CPU hotspots, and JVM internals.

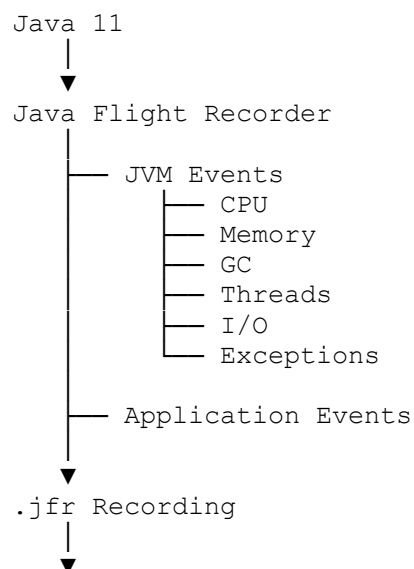
3. Production Diagnostics

Because of its low-overhead design, JFR can be useful for diagnosing performance problems in production environments.

4. Custom Events

Developers can instrument their own application-specific events using `jdk.jfr.Event`.

Key Point



JFR Analysis



Performance Optimization

In short: Java Flight Recorder (JFR) in Java 11 provides a powerful, low-overhead mechanism for **recording and analyzing JVM and application runtime behavior**, making it especially useful for performance troubleshooting and production diagnostics.

Java 11 — ZGC Experimental

Java Version: 11

Year: 2018

Feature: Z Garbage Collector (ZGC) — Experimental

UniqueClassName: ZGC

Definition

ZGC (Z Garbage Collector) is a low-latency garbage collector introduced as an **experimental feature in JDK 11**.

Its primary goal is to perform garbage collection while keeping **pause times extremely short**, even when working with very large Java heaps.

ZGC was designed for applications where long GC pauses are undesirable, such as:

- High-throughput server applications
- Large-memory applications
- Real-time-ish services
- Low-latency systems
- Applications with large heaps

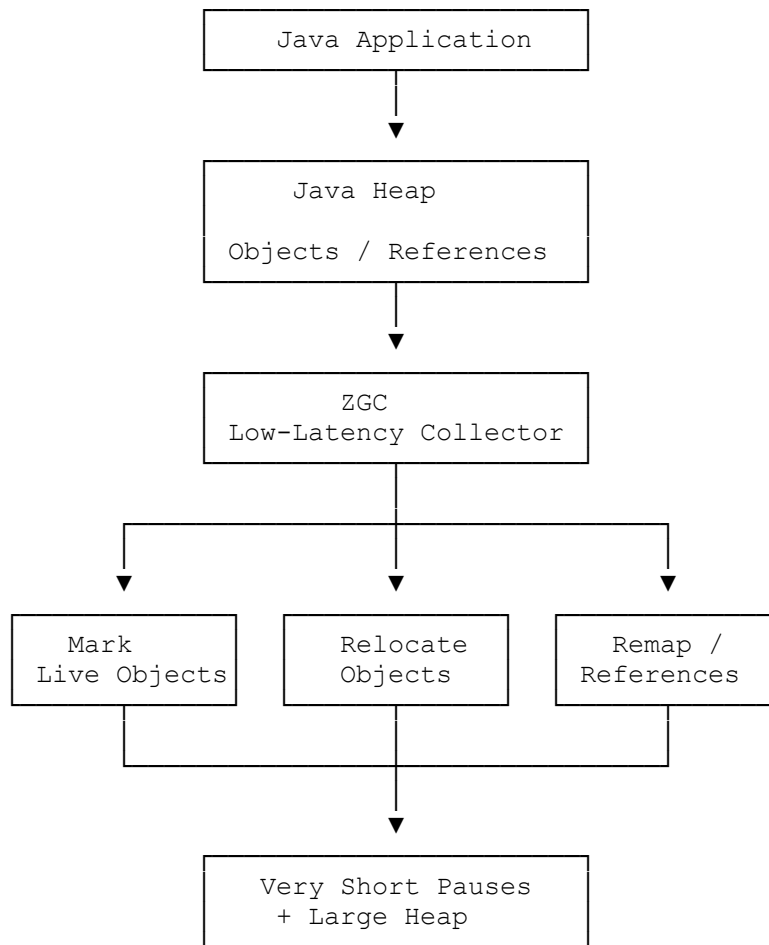
In JDK 11, ZGC was **experimental**, so it was not the default garbage collector.

It could be enabled with:

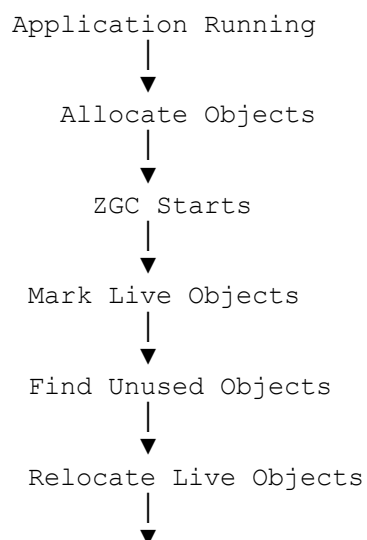
`-XX:+UnlockExperimentalVMOptions -XX:+UseZGC`

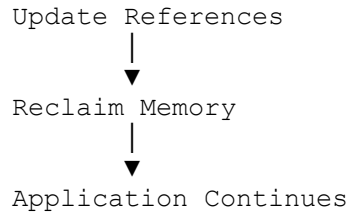
Note: ZGC has evolved significantly since Java 11. Modern JDKs have different ZGC options and behavior, so Java 11 examples should not be assumed to apply unchanged to current JDK releases.

ZGC Flowchart



How ZGC Works — Simplified





A key characteristic of ZGC is that much of the GC work is performed **concurrently with application threads**, reducing the need for long stop-the-world pauses.

Java Code Example

The Java application itself does not need special ZGC APIs. You enable ZGC when starting the JVM.

ZGC.java

```
public class ZGC {  
    public static void main(String[] args) throws InterruptedException {  
        System.out.println("Application started.");  
  
        for (int i = 1; i <= 10; i++) {  
            // Allocate objects to create heap activity  
            byte[] data = new byte[10 * 1024 * 1024];  
  
            System.out.println(  
                "Allocated approximately 10 MB - iteration " + i  
            );  
  
            Thread.sleep(500);  
  
            // Make the object eligible for garbage collection  
            data = null;  
        }  
  
        System.out.println("Application finished.");  
  
        Thread.sleep(2000);  
    }  
}
```

Compile

```
javac ZGC.java
```

Run with Java 11 ZGC

```
java -XX:+UnlockExperimentalVMOptions -XX:+UseZGC ZGC
```

The important part is:

```
-XX:+UnlockExperimentalVMOptions  
-XX:+UseZGC
```

The first option unlocks experimental JVM options, and the second selects ZGC as the garbage collector.

Example with GC Logging

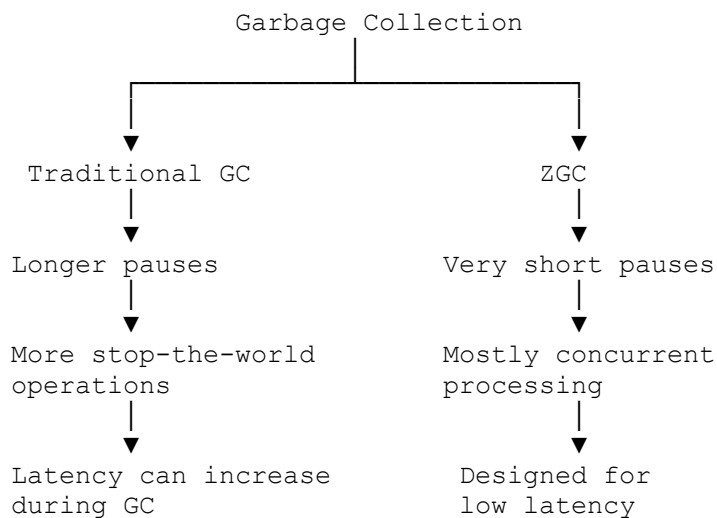
You can also enable GC logging to observe ZGC activity.

```
java \  
-XX:+UnlockExperimentalVMOptions \  
-XX:+UseZGC \  
-Xlog:gc \  
ZGC
```

This allows you to see garbage-collection information in the console.

For Java 11, the unified JVM logging system introduced in Java 9 is used through -Xlog.

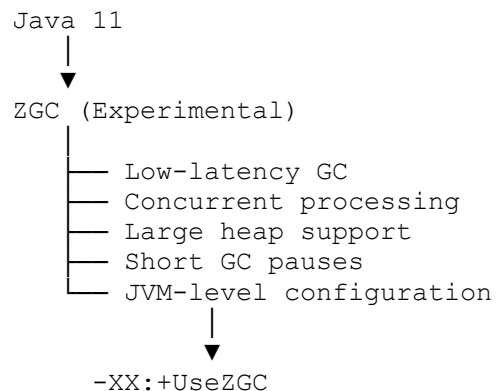
ZGC vs Traditional GC



Important Characteristics of ZGC in Java 11

Characteristic	ZGC
Introduced	JDK 11
Status in JDK 11	Experimental
Main goal	Low GC latency
Package/API	No application-level ZGC API required
Activation	JVM command-line option
Concurrent processing	Yes
Large heap support	Yes
Default GC in Java 11	No

Key Point



In short: ZGC in Java 11 was an experimental, low-latency garbage collector designed to minimize GC pause times by performing much of its work concurrently with application execution.